Py Rhythm

1. Project Overview

Py Rhythm is a rhythm-based music game developed using Python. In this game, players must press keys in sync with music beats as notes fall toward a judgement line. The player needs to hit the notes at the correct timing to gain points and maintain a combo.

The game will include different note types such as ***Tap*** *(tap the keyboard)****,*** ***Hold*** *(hold the keyboard)****, and Slide*** *(check is the mouse in the left or right of screen),*

each requiring different player actions. The scoring system will evaluate the player's timing accuracy and display results such as Perfect, Good, or Miss.

The project also focuses on collecting gameplay statistics such as player accuracy, score, and reaction time. These statistics will later be analyzed to understand player performance.

2. Project Review

Py Rhythm aims to create a ***simplified rhythm game using Python*** *for educational purposes.*

The main goals of this project are:

- *To practice* ***game development using Python***
- *To apply* ***Object-Oriented Programming concepts***
- *To collect* ***gameplay statistics for analysis***

Compared to existing rhythm games, Py Rhythm focuses more on learning programming concepts and data analysis rather than creating a full commercial game.

3. Programming Development

3.1 Game Concept

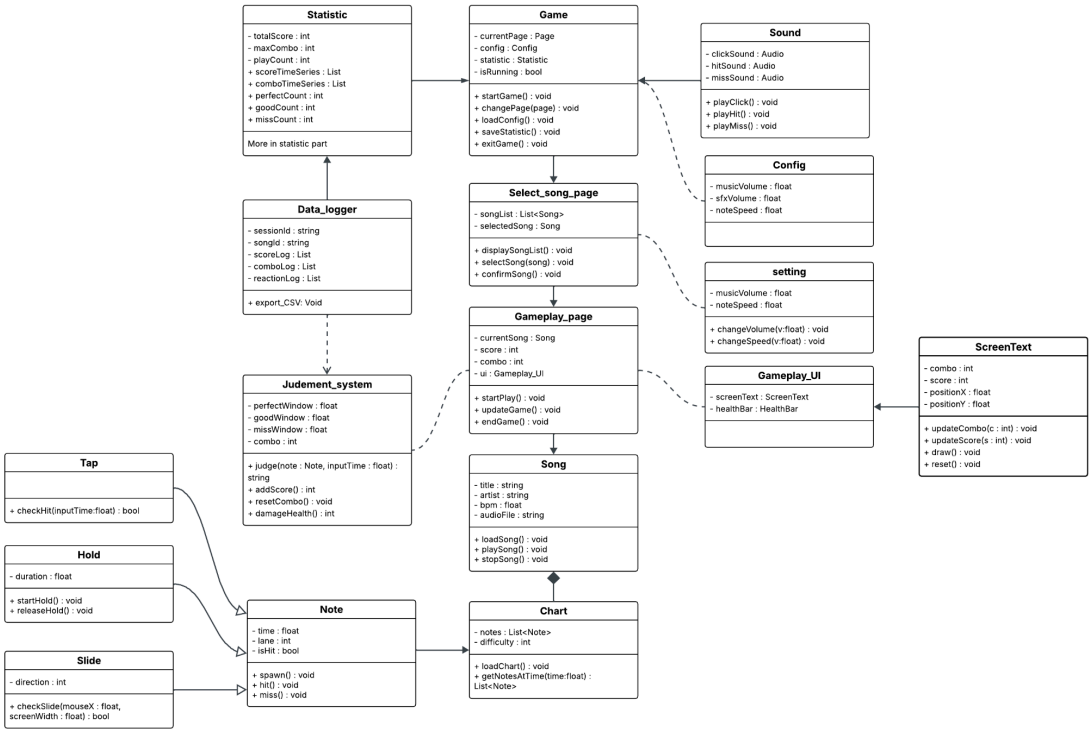
Py Rhythm is a rhythm game where notes move toward a judgement line while music is playing. The player must press the correct keys when the notes reach the target area.

Game Mechanics

1. A song is selected from the song menu.
2. Notes appear based on the song's chart.
3. Notes move toward the judgement line.
4. The player presses keys to hit the notes.

3.2 Object-Oriented Programming Implementation

The game will be developed using Object-Oriented Programming (OOP). Several classes will be used to organize the game structure and functionality. And this is my second update version of UML class diagram



3.3 Algorithms Involved

Several algorithms and programming techniques will be used in the game.

Timing Judgement Algorithm

The system compares the player's input time with the note timing to determine whether the result is Perfect, Good, or Miss.

Event-driven Input Handling

The game listens for keyboard inputs and processes them when players interact with notes.

Score Calculation

The score is calculated based on timing accuracy and combo multipliers.

4. Statistical Data (Prop Stats)

4.1 Data Features

Examples of data that I will track include:

1. ***Player score:***
   The player's score will be recorded every 10 second. This data will be used to analyze how the score evolves over time during gameplay.

2. ***Player combo in time series :***
   The player's combo will be recorded whenever the combo changes. This helps identify the exact moments when the player misses and loses their combo.

3. ***Number of Perfect, Good, and Miss hits for each song:***
   The total number of Perfect, Good, and Miss results will be recorded at the end of each chart. This data is used to evaluate the player's overall performance for each song.

4. ***Type of note that player miss:***
   The type of note (Tap, Hold, Slide) that causes a miss will be recorded each time the player breaks a combo. The data will be analyzed as percentages to determine which note types are most difficult for the player.

5. ***Reaction time between note timing and player input:***
   The reaction time between the note timing and the player's input will be recorded each time a note is hit. This data will be used to compare the player's timing accuracy with the perfect timing.

4.2 Data Recording Method

*The statistical data will be stored in a **CSV file** after each gameplay session. Using CSV files allows the data to be easily analyzed using tools such as Python.*

4.3 Data Analysis Report

1. Player Score Over Time

*Graph: **BarChart ,** Reason: Used for data to show how the score changes during gameplay.*

2. Player Combo Over Time

*Graph: **Line Chart,** Reason: Shows changes in combo over time and clearly highlights when the combo drops (miss).*

3. Perfect, Good, and Miss Hits

*Graph: **Table,** Reason: Used to compare the number of each result and evaluate overall accuracy.*

4. Types of Notes Missed (Tap, Hold, Slide)

*Graph: **Pie Chart,** Reason: Used to compare categories and identify which note type is most difficult.*

5. Reaction Time

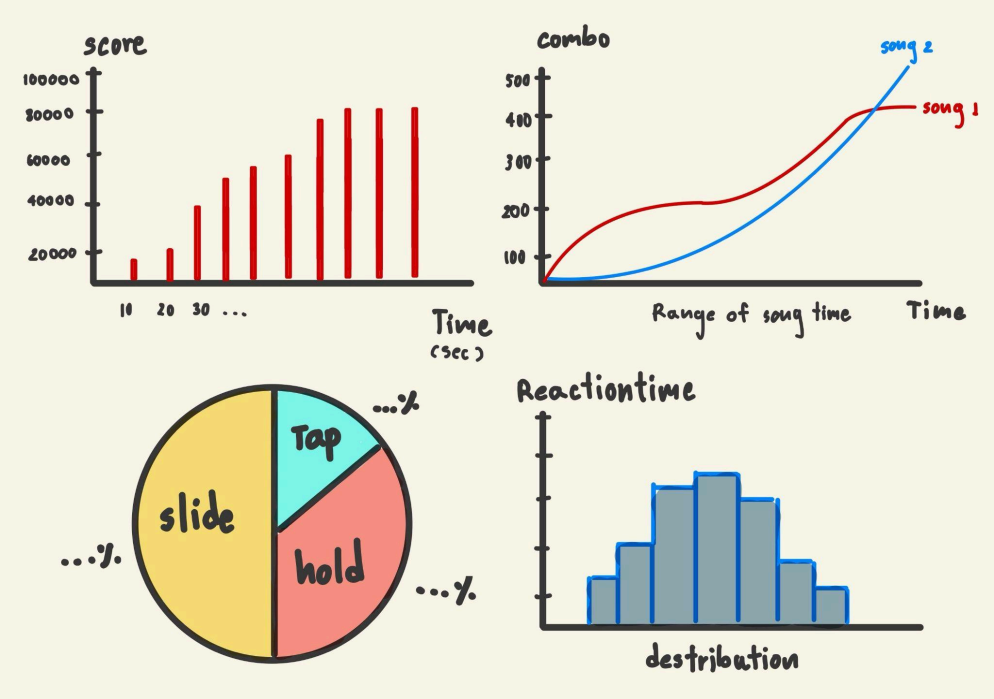
*Graph: **Histogram,** Reason: Used to show the distribution of reaction time and analyze whether the player hits early, late, or close to perfect timing.*

4.4 Update more about statistical Data

	Why It good to have this data	*How will obtain 100 value*	*Which variable or class I will collect*	*How will I display data*
Player Score every 5 second	To check how many score player get over the time, good to see performance in detail	Collect each time the score change (get more than 300 rows per session per song)	Will be collect in Data_logger class detail in my UML	Bar chart
Player Combo Over Time	To check how many combo player get over the time,to see when or what part of chart make player miss, good to improve future chart	Collect each time the combo change (get more than 300 rows per session per song)	Will be collect in Data_logger class detail in my UML	Line chart
Perfect, Good, and Miss Hits in all song	To check how many perfect good or miss player get for each song, good to see over all performance and could be show to player after session	Collect each time player get *Perfect, Good, or Miss*	Will be collect in Data_logger class detail in my UML	Table
Types of Notes Missed	To check what the type of note is hardest for the player, good to see how each type of note effect the player and also to improve future chart	Collect each time the player miss (get more than around 20 rows per session per song depend on player performance)	Will be collect in Data_logger class detail in my UML	Pie chart
Reaction Time	To check what the delay or early time player react, good to improve note speed setting or make new feature	Collect each time player don't get perfect combo (get more than around 50 rows per session per song depend on player performance)	Will be collect in Data_logger class detail in my UML	Histogram maybe normal distribution

	Feature name	*Graph objective*	*Graph type*	*X - axis*	*Y - axis*
Graph 1	*Player Score every 5 second*	*Show how the player's score changes over time*	*Line graph*	*Time (seconds)*	*Player score*
Graph 2	*Player Combo Over Time*	*Compare the player's combo at different time intervals*	*Bar graph*	*Time (or time intervals)*	*Combo count*
Graph 3	*Types of Notes Missed*	*Show the proportion of each type of missed notes*	*Pie graph*	*None*	*None*
Graph 4	*Reaction Time*	*Analyze the distribution of players' reaction times*	Normal distribution	*Histogram (Normal distribution)*	*Reaction time (seconds or milliseconds)*
Graph 5	*Perfect, Good, and Miss Hits in all song*	*Show overall performance*	Table	*table*	*table*

And this is my hand-drawn graph to see overall how it going to look like



5. Project Planning Timeline

Week	Week	Task
1	2	Proposal submission / Project initiation
3	4	Research rhythm game mechanics and project design
5	6	Design UML diagram and Start game structure
7	8	Implement core classes and basic game framework
9	10	Develop gameplay mechanics (note system, input detection)
11	12	Implement scoring system and statistical data recording
13	14	Submission week

6. Document version

Version: 1.0
Date: 25 March 2026
Editor: phutharak wongpitak

Date	Name	Description of Revision, Feedback, Comments
18/3	Nattanan	You did really well on expanding the class, if in my opinion, I might not make each page each class but that will do too. However, your arrows are still wrong in many points. I point out some crucial one already but more than that try study with the source i sent you For the data part I want you to be more detailed. Reminder!! The music and SFX have their own copyright! Make sure that you use the music that allows you to use (check out free assets). Don't forget to manage the license and copyright properly